MINISTERE DE L'ENSEIGNEMENT SUPERIEUR, DE LA RECHERCHE ET DE L'INNOVATION

****

ECOLE POLYTECHNIQUE DE OUAGADOUGOU

****

DIRECTION GENERALE

INSTITUT DU GENIE INFORMATIQUE ET DES TELECOMMUNICATIONS

****

Département : Génie Informatique

BURKINA FASO

La Patrie ou Ia Mart
Nous Vaincrons

Module ; Ingénierie de la cryptographie

THEME : Chiffrement authentifié et dérivation de clés

Nom de l'Etudiant :

CISSE Abdoul Wahabou

Enseignant :

Dr Zakaria Sawadogo

Introduction

Dans ce TP, j'ai construit un mini-coffre de fichiers en Python (coffre.py). Il chiffre un fichier avec un mot de passe. La clé est dérivée avec Argon2id, puis des sous-clés sont produites avec HKDF. Le chiffrement utilise AES-GCM, un mode qui protège à la fois la confidentialité et l'intégrité des données.

Le fichier chiffré suit un format versionné. Les 44 premiers octets forment l'en-tête : ils restent en clair, mais ils sont authentifiés avec le nom du fichier (données associées). Neuf tests automatiques vérifient les propriétés de sécurité du coffre.

Champ	Taille	Rôle
Identifiant CFR1	4 octets	Reconnaître le format et sa version
Paramètres Argon2id (t, m, p)	12 octets	Redériver la clé et durcir plus tard
Sel	16 octets	Rendre la dérivation unique pour ce fichier
Nonce	12 octets	Valeur unique pour ce chiffrement
Chiffré + étiquette	variable	Données protégées par AES-GCM

Fichiers et exécution du code

Fichier	Rôle
coffre.py	Le coffre : cle_maitresse (Argon2id), sous_cle (HKDF), chiffrer, dechiffrer, rechiffrer (AES-GCM)
test_coffre.py	Les 9 tests pytest qui prouvent les propriétés de sécurité
compte_rendu_tp1.pdf	Ce compte rendu

1. Préparer l'environnement (une seule fois). On crée un environnement Python isolé et on installe les bibliothèques :

```
python3 -m venv ~/ec1
source ~/ec1/bin/activate
pip install cryptography argon2-cffi pytest
```

2. Lancer le coffre. Le programme chiffre un message, le chiffre une seconde fois, puis le déchiffre. Il affiche la taille du fichier chiffré (message + 60 octets), l'identifiant CFR1 et le fait que deux chiffrements ne sont jamais identiques :

```
python3 coffre.py
```

3. Lancer les tests. pytest exécute toutes les fonctions test_… de test_coffre.py. Le résultat attendu est « 9 passed » :

```
pytest -v
```

Questions de réflexion

Question 1 - Pourquoi le sel est-il stocké en clair ? Sa publication affaiblit-elle la sécurité ?

Le sel est enregistré en clair dans l'en-tête du fichier, parce qu'il est nécessaire pour retrouver la même clé lors du déchiffrement. Sans lui, on ne peut pas recalculer la clé à partir du mot de passe.

Sa publication n'affaiblit pas la sécurité : le sel n'est pas secret, c'est le mot de passe qui protège les données. Le sel sert à créer une clé différente pour chaque fichier, même si on utilise le même mot de passe. Il empêche aussi un attaquant d'utiliser des calculs préparés à l'avance pour deviner

les mots de passe. Grâce à Argon2id, qui est lent et gourmand en mémoire, l'attaquant doit refaire un calcul coûteux pour chaque fichier et pour chaque mot de passe testé.

Question 2 - Pourquoi n'y a-t-il aucun risque de répétition du nonce de 96 bits, même pour des millions de fichiers ?

Avec AES-GCM, il ne faut jamais utiliser le même nonce avec la même clé pour chiffrer deux messages différents. Sinon, un attaquant pourrait obtenir des informations sur les messages et affaiblir leur protection.

Dans notre programme, chaque fichier reçoit un nouveau sel, donc une nouvelle clé, ainsi qu'un nouveau nonce. Même si deux fichiers avaient le même nonce, leurs clés seraient différentes : cela ne poserait donc pas de problème. De plus, il existe un très grand nombre de nonces possibles (2^{96}, soit environ 8×10^{28}). La probabilité d'en tirer deux identiques parmi quelques millions de fichiers est extrêmement faible.

Question 3 - Que se passerait-il si le nom du fichier n'était pas dans les données associées ?

Prenons un exemple. Deux fichiers, releve_bancaire.txt et liste_courses.txt, sont protégés avec le même mot de passe. Un pirate qui a accès au stockage, mais qui n'a pas le mot de passe, échange leurs noms.

Si le nom est dans les données associées, le programme détecte le problème et refuse le déchiffrement (exception InvalidTag). Si le nom n'y est pas, le programme déchiffre le contenu sans erreur, car le contenu n'a pas été modifié et rien ne vérifie le lien entre le nom et le contenu. L'utilisateur croit alors ouvrir son relevé bancaire alors qu'il voit la liste de courses. C'est un problème de sécurité : le pirate trompe l'utilisateur sans connaître le mot de passe et sans lire le contenu, et l'utilisateur pourrait prendre une décision à partir d'un document qui n'est pas le bon.

Question 4 - Pourquoi l'en-tête lui-même est-il authentifié ?

L'en-tête est stocké en clair, donc toute personne qui a accès au fichier peut le modifier. Il est authentifié (passé en données associées) pour que toute modification soit détectée : le déchiffrement échoue alors avec InvalidTag. Cela concerne la version, les paramètres Argon2id, le sel et le nonce.

Sans cette protection, un attaquant pourrait modifier les paramètres d'Argon2id. Avec une mémoire énorme, il pourrait consommer toute la mémoire de la machine et la ralentir ou la bloquer. Avec des valeurs très faibles, il pourrait affaiblir la dérivation de la clé. Le principe à retenir est que tout ce qui est lu avant le déchiffrement doit être protégé.

Une limite doit être notée : l'étiquette n'est vérifiée qu'après le calcul d'Argon2id. Une mémoire excessive serait donc consommée avant la détection. Une précaution simple est de vérifier que les paramètres restent dans des bornes raisonnables avant de lancer Argon2id.

Question 5 - Pourquoi ne doit-on jamais afficher la raison précise d'un échec de déchiffrement ?

Si le programme affichait des messages différents (mot de passe incorrect, fichier modifié, nom invalide), un pirate pourrait essayer plusieurs fois et utiliser ces messages pour comprendre ce qui ne va pas. Le programme servirait d'oracle : chaque réponse donnerait une information qui l'aiderait à préparer une attaque.

Avec AES-GCM, toutes ces causes produisent la même exception, InvalidTag. L'interface doit donc afficher toujours le même message, par exemple « Déchiffrement impossible », sans donner d'indice sur la cause. Les détails techniques peuvent être gardés dans un journal réservé à l'administrateur. Cela renforce la sécurité du coffre.

Conclusion

On ne réinvente aucun algorithme : on assemble correctement des briques éprouvées (Argon2id, HKDF, AES-GCM). La sécurité vient de détails précis : un sel et un nonce neufs à chaque fichier, des données associées bien choisies, un format versionné et des erreurs qui ne révèlent rien.